Maharashtra Education Society's
Institute of Management and Career Courses (IMCC), Pune
AUTONOMOUS

Academic Year 2026-27
FY MCA- SEM I
IT12 Data Structure and Algorithms
Practice Assignment1

Roll No: 2601196
Name: Ashutosh Uttekar

```
class Node:
    """Class to represent a node in the Linked List."""
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    # 1. Create Linked List / Append Node
    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
            last = last.next
        last.next = new_node

    # 2. Traverse and print the node values
    def traverse(self):
        if not self.head:
            print("Linked List is empty.")
            return
        current = self.head
        elements = []
        while current:
            elements.append(str(current.data))
            current = current.next
        print(" -> ".join(elements) + " -> None")
```

```
    # 3. Insert node at a specific position (0-indexed)
def insert_at_position(self, data, position):
    new_node = Node(data)

    # Inserting at the head (position 0)
    if position == 0:
        new_node.next = self.head
        self.head = new_node
        return

    current = self.head
    for _ in range(position - 1):
        if current is None:
            print("Position out of bounds.")
            return
        current = current.next

    if current is None:
        print("Position out of bounds.")
        return

    new_node.next = current.next
    current.next = new_node

# 4. Find Middle node and print its value (Using Tortoise and Hare algorithm)
def find_middle(self):
    if not self.head:
        print("List is empty, no middle node.")
        return None

    slow = self.head
    fast = self.head

    # Fast pointer moves 2 steps, slow pointer moves 1 step
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    print(f"Middle node value: {slow.data}")
    return slow.data

# 5. Delete node (by value)
def delete_node(self, key):
```

```
        current = self.head

    # Case 1: If the head node itself holds the key
    if current and current.data == key:
        self.head = current.next
        current = None
        return

    # Case 2: Search for the key to be deleted
    prev = None
    while current and current.data != key:
        prev = current
        current = current.next

    # If key was not present in the linked list
    if current is None:
        print(f"Node with value {key} not found.")
        return

    # Unlink the node from linked list
    prev.next = current.next
    current = None

# 6. Reverse list
def reverse(self):
    prev = None
    current = self.head

    while current:
        next_node = current.next  # Store next node
        current.next = prev       # Reverse the link
        prev = current            # Move prev and current one step forward
        current = next_node

    self.head = prev

# 7. Calculate the sum of every two consecutive node values
def sum_consecutive(self):
    if not self.head or not self.head.next:
        print("Not enough nodes to calculate consecutive sums.")
        return

    sums = []
```

```
        current = self.head

        while current.next:
            consecutive_sum = current.data + current.next.data
            sums.append(consecutive_sum)
            current = current.next

        print("Sum of consecutive nodes:", sums)

# ==========================================
# Driver Code to test the operations
# ==========================================
if __name__ == "__main__":
    ll = LinkedList()

    print("1 & 2. Creating and Traversing Linked List:")
    for val in [10, 20, 30, 40, 50]:
        ll.append(val)
    ll.traverse()

    print("\n3. Inserting 25 at position 2:")
    ll.insert_at_position(25, 2)
    ll.traverse()

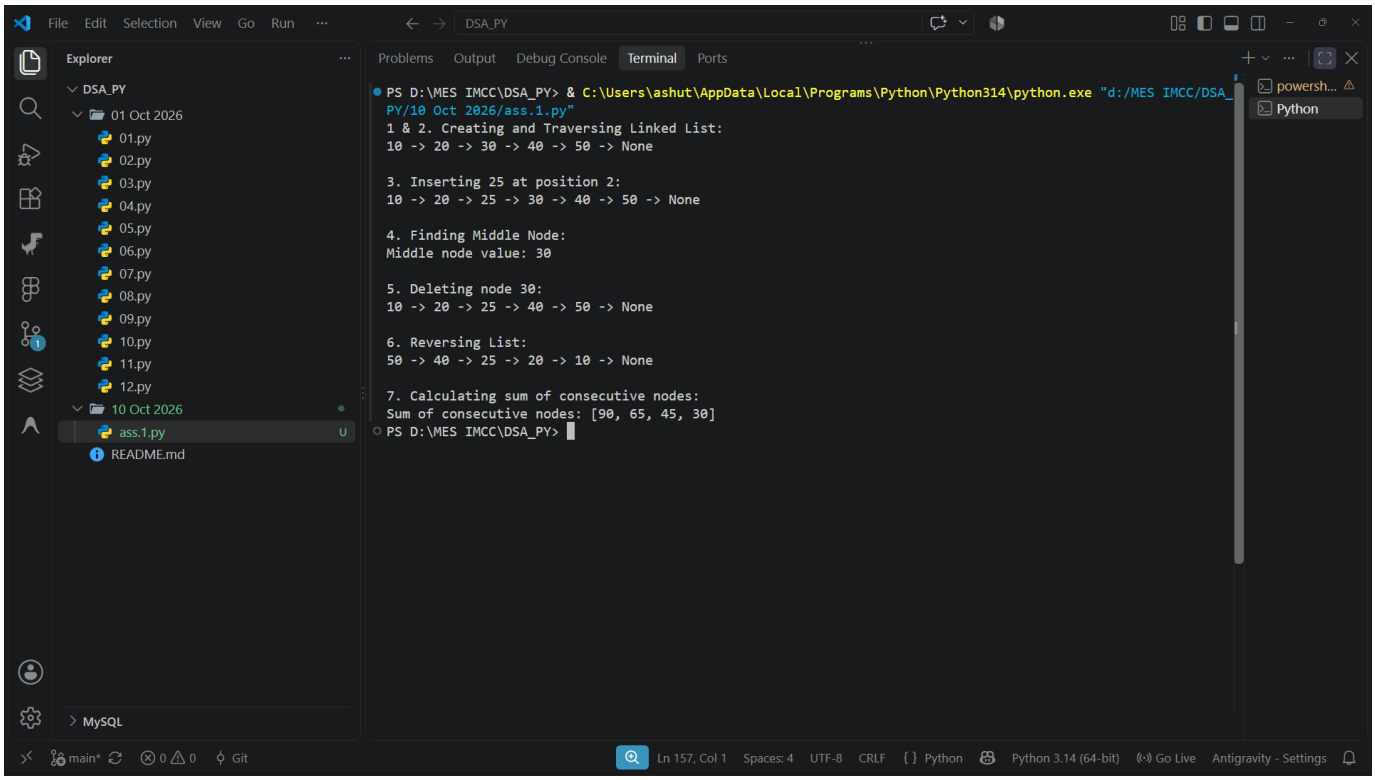
    print("\n4. Finding Middle Node:")
    ll.find_middle()

    print("\n5. Deleting node 30:")
    ll.delete_node(30)
    ll.traverse()

    print("\n6. Reversing List:")
    ll.reverse()
    ll.traverse()

    print("\n7. Calculating sum of consecutive nodes:")
    ll.sum_consecutive()
```

OUTPUT


File Edit Selection View Go Run
DSA_PY
Explorer
DSA_PY
01 Oct 2026
01.py
02.py
03.py
04.py
05.py
06.py
07.py
08.py
09.py
10.py
11.py
12.py
10 Oct 2026
ass.1.py
README.md
MySQL
Problems Output Debug Console Terminal Ports
PS D:\MES IMCC\DSA_PY> & C:\Users\ashut\AppData\Local\Programs\Python\Python314\python.exe "d:/MES IMCC/DSA_PY/10 Oct 2026/ass.1.py"
1 & 2. Creating and Traversing Linked List:
10 -> 20 -> 30 -> 40 -> 50 -> None
3. Inserting 25 at position 2:
10 -> 20 -> 25 -> 30 -> 40 -> 50 -> None
4. Finding Middle Node:
Middle node value: 30
5. Deleting node 30:
10 -> 20 -> 25 -> 40 -> 50 -> None
6. Reversing List:
50 -> 40 -> 25 -> 20 -> 10 -> None
7. Calculating sum of consecutive nodes:
Sum of consecutive nodes: [90, 65, 45, 30]
PS D:\MES IMCC\DSA_PY>
powersh...
Python
main*
Git
Ln 157, Col 1
Spaces: 4
UTF-8
CRLF
Python
Python 3.14 (64-bit)
Go Live
Antigravity - Settings